

padarray Function Test Results

Test Case 1: Default Zero Padding

- **Input Matrix:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
- **Padding Size:** $[1 \ 1]$
- **Option:** Default (zero padding)
- **Command:** `padarray(A,[1 1])`

Result: Matrix padded with zeros on all sides

```
0. 0. 0. 0.
0. 1. 2. 0.
0. 3. 4. 0.
0. 0. 0. 0.
```

Test Case 2: Explicit Zero Padding

- **Input Matrix:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
- **Padding Size:** $[1 \ 1]$
- **Option:** "zeros"
- **Command:** `padarray(A,[1 1],"zeros")`

Result: Same output as zero padding

```
0. 0. 0. 0.
0. 1. 2. 0.
0. 3. 4. 0.
0. 0. 0. 0.
```

Test Case 3: Constant Padding

- **Input Matrix:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
- **Padding Size:** $[1 \ 1]$
- **Option:** 5

- **Command:** `padarray(A,[1 1],5)`

Result: Borders filled with value 5

```
5. 5. 5. 5.
5. 1. 2. 5.
5. 3. 4. 5.
5. 5. 5. 5.
```

Test Case 4: Replicate Padding

- **Input Matrix:** `A = [1 2; 3 4]`
- **Padding Size:** `[1 1]`
- **Option:** `"replicate"`
- **Command:** `padarray(A,[1 1],"replicate")`

Result: Edge values repeated outward

```
1. 1. 2. 2.
1. 1. 2. 2.
3. 3. 4. 4.
3. 3. 4. 4.
```

Test Case 5: Circular Padding

- **Input Matrix:** `A = [1 2; 3 4]`
- **Padding Size:** `[1 1]`
- **Option:** `"circular"`
- **Command:** `padarray(A,[1 1],"circular")`

Result: Values wrapped from opposite edges

```
4. 3. 4. 3.
2. 1. 2. 1.
4. 3. 4. 3.
2. 1. 2. 1.
```

Test Case 6: Reflect Padding

- **Input Matrix:** A = [1 2 3; 4 5 6; 7 8 9]
- **Padding Size:** [1 1]
- **Option:** "reflect"
- **Command:** padarray(A,[1 1],"reflect")

Result: Matrix reflected excluding edge pixels

```
5. 4. 5. 6. 5.  
2. 1. 2. 3. 2.  
5. 4. 5. 6. 5.  
8. 7. 8. 9. 8.  
5. 4. 5. 6. 5.
```

Test Case 7: Symmetric Padding

- **Input Matrix:** Same as above 3×3 matrix
- **Padding Size:** [1 1]
- **Option:** "symmetric"
- **Command:** padarray(A,[1 1],"symmetric")

Result: Matrix reflected including edge pixels

```
1. 1. 2. 3. 3.  
1. 1. 2. 3. 3.  
4. 4. 5. 6. 6.  
7. 7. 8. 9. 9.  
7. 7. 8. 9. 9.
```

Test Case 8: Non-Square Padding

- **Input Matrix:** A = [1 2 3; 4 5 6]
- **Padding Size:** [1 3]

- **Option:** "replicate"
- **Command:** padarray(A,[1 3],"replicate")

Result: Unequal row and column padding applied

```

1. 1. 1. 1. 2. 3. 3. 3. 3.
1. 1. 1. 1. 2. 3. 3. 3. 3.
4. 4. 4. 4. 5. 6. 6. 6. 6.
4. 4. 4. 4. 5. 6. 6. 6. 6.

```

Test Case 9: Single Pixel Image

- **Input Matrix:** A = 7
- **Padding Size:** [2 2]
- **Option:** 0
- **Command:** padarray(A,[2 2],0)

Result: Center value preserved with zero border

```

0. 0. 0. 0. 0.
0. 0. 0. 0. 0.
0. 0. 7. 0. 0.
0. 0. 0. 0. 0.
0. 0. 0. 0. 0.

```

Test Case 10: No Padding

- **Input Matrix:** A = [1 2; 3 4]
- **Padding Size:** [0 0]
- **Option:** Default
- **Command:** padarray(A,[0 0])

Result: Original matrix returned unchanged

```

1. 2.
3. 4.

```

Test Case 11: Invalid Negative Padding

- **Input Matrix:** A = [1 2; 3 4]
- **Padding Size:** [-1 1]
- **Option:** "replicate"
- **Command:** padarray(A,[-1 1],"replicate")

Result: Error handled correctly - "padarray: padsize values must be non-negative."

Test Case 12: Invalid Reflect Padding

- **Input Matrix:** A = [1 2; 3 4]
- **Padding Size:** [2 2]
- **Option:** "reflect"
- **Command:** padarray(A,[2 2],"reflect")

Result: Error handled correctly - "padarray: unsupported padding type."